# Prelucrarea Imaginilor

Prof. Dr. Călin-Adrian POPA

## Cursul 9

24 Noiembrie 2025

# 9 Video classification

- *video classification* represents the task of identifying actions in videos, as opposed to image classification, where only the most important object in the image has to be identified
- in video classification, the models have access to not only the appearance information present in single, static images, but also their complex temporal evolution

# 9 Video classification



Images: Recognize **objects**

Dog
**Cat**
Fish
Truck



Videos: Recognize **actions**

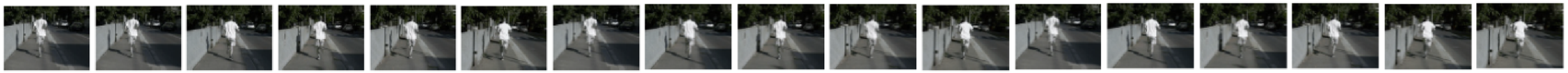
Swimming
**Running**
Jumping
Eating
Standing

# 9 Video classification

- the problem is that videos are big
- for example, an uncompressed video (3 bytes per pixel) in SD ($640 \times 480$) resolution is $\sim 1.5$ GB per minute, and in HD ($1920 \times 1080$) resolution is $\sim 10$ GB per minute
- the solution is to train on short clips, which have low fps and low spatial resolution, for example, $T = 16$, $H = W = 112$ (a 3.2 seconds video at 5 fps is just 588 KB)

# 9 Video classification

**Raw video**: Long, high FPS



**Training**: Train model to classify short **clips** with low FPS



**Testing**: Run model on different clips, average predictions



# 9.1 Large-scale video classification with CNNs

- unlike images, which can be cropped and rescaled to a fixed size, videos vary widely in temporal extent and cannot be easily processed with a fixed-sized architecture
- thus, every video is treated as a bag of short, fixed-sized clips
- since each clip contains several contiguous frames in time, we can extend the connectivity of the network in the time dimension to learn spatio-temporal features
- there are multiple options for the precise details of the extended connectivity and we describe three broad connectivity pattern categories (Early Fusion, Late Fusion, and Slow Fusion) below
- afterwards, we describe a multiresolution architecture for addressing the computational efficiency

# 9.1 Large-scale video classification with CNNs

- several approaches to fusing information across temporal domain are investigated (Figure 1):
  - the fusion can be done early in the network by modifying the first layer convolutional filters to extend in time, or
  - it can be done late by placing two separate single-frame networks some distance in time apart and fusing their outputs later in the processing
- we first describe a baseline single-frame CNN and then discuss its extensions in time according to different types of fusion

# 9.1 Large-scale video classification with CNNs

- **Single Frame**: a Single Frame baseline architecture is used to understand the contribution of static appearance to the classification accuracy
- this network accepts inputs of size $170 \times 170 \times 3$ pixels
- using shorthand notation, the full architecture is $C(96, 11, 3)$-$N$-$P$-$C(256, 5, 1)$-$N$-$P$-$C(384, 3, 1)$-$C(384, 3, 1)$-$C(256, 3, 1)$-$P$-$FC(4096)$-$FC(4096)$, where $C(d, f, s)$ indicates a convolutional layer with $d$ filters of spatial size $f \times f$, applied to the input with stride $s$, $FC(n)$ is a fully-connected layer with $n$ nodes
- all pooling layers $P$ pool spatially in non-overlapping $2 \times 2$ regions and all normalization layers $N$ are defined as described in the AlexNet paper
- the final layer is connected to a softmax classifier with dense connections

# 9.1 Large-scale video classification with CNNs

- **Early Fusion**: the Early Fusion extension combines information across an entire time window immediately on the pixel level
- this is implemented by modifying the filters on the first convolutional layer in the single-frame model by extending them to be of size $11 \times 11 \times 3 \times T$ pixels, where $T$ is some temporal extent ($T = 10$ is used, or approximately a third of a second)
- the early and direct connectivity to pixel data allows the network to precisely detect local motion direction and speed

# 9.1 Large-scale video classification with CNNs

- **Late Fusion**: the Late Fusion model places two separate single-frame networks (as described above), up to the last convolutional layer $C(256, 3, 1)$ with shared parameters a distance of 15 frames apart and then merges the two streams in the first fully-connected layer
- therefore, neither single-frame tower alone can detect any motion, but the first fully-connected layer can compute global motion characteristics by comparing outputs of both towers

# 9.1 Large-scale video classification with CNNs

- **Slow Fusion**: the Slow Fusion model is a balanced mix between the two approaches that slowly fuses temporal information throughout the network, such that higher layers get access to progressively more global information in both spatial and temporal dimensions
- this is implemented by extending the connectivity of all convolutional layers in time and carrying out temporal convolutions in addition to spatial convolutions to compute activations
- in the used model, the first convolutional layer is extended to apply every filter of temporal extent $T = 4$ on an input clip of 10 frames through valid convolution with stride 2 and produces 4 responses in time
- the second and third layers above iterate this process with filters of temporal extent $T = 2$ and stride 2
- thus, the third convolutional layer has access to information across all 10 input frames

# 9.1 Large-scale video classification with CNNs

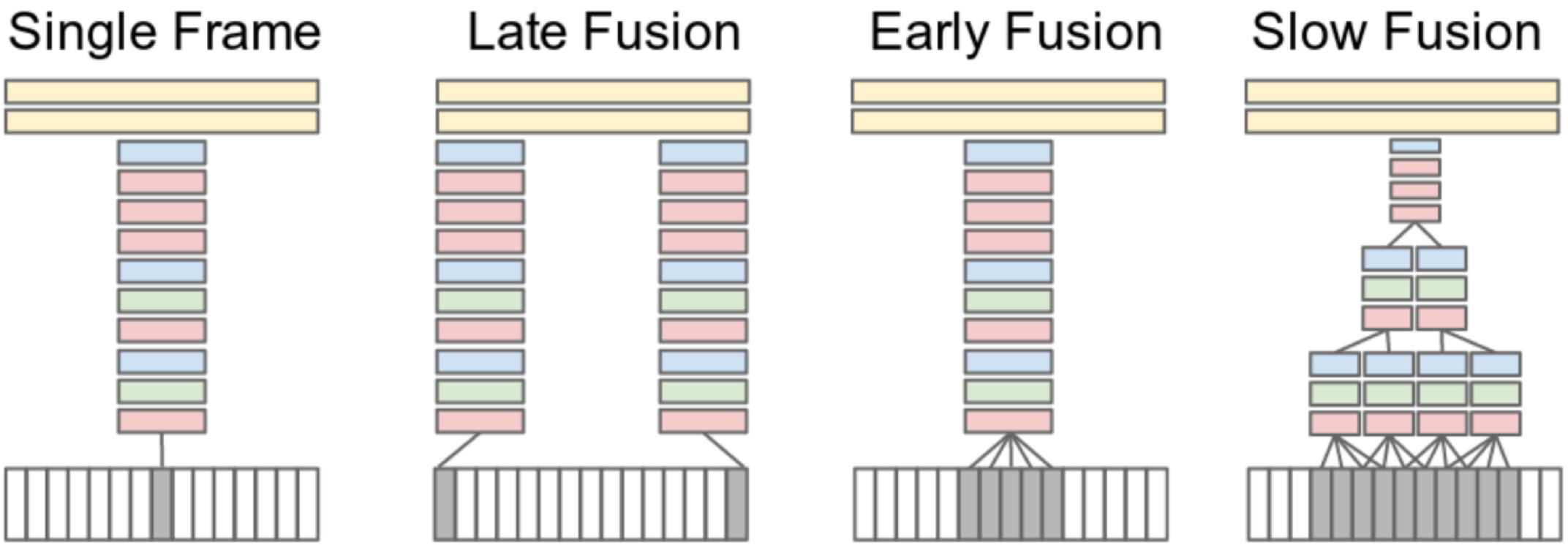



Figure 1: Explored approaches for fusing information over temporal dimension through the network. Red, green, and blue boxes indicate convolutional, normalization, and pooling layers, respectively. In the Slow Fusion model, the depicted columns share parameters.

# 9.1 Large-scale video classification with CNNs

- **Multiresolution CNNs**: the runtime performance is a critical component to the ability to experiment with different architecture and hyperparameter settings
- this motivates approaches for speeding up the models, while still retaining their performance
- there are multiple fronts to these endeavors, including improvements in hardware, weight quantization schemes, better optimization algorithms, and initialization strategies, but we focus on changes in the architecture that enable faster running times without sacrificing performance

# 9.1 Large-scale video classification with CNNs

- one approach to speeding up the networks is to reduce the number of layers and neurons in each layer, but this is found to consistently lower the performance
- instead of reducing the size of the network, further experiments were conducted on training with images of lower resolution
- however, while this improved the running time of the network, the high-frequency detail in the images proved critical to achieving good accuracy

# 9.1 Large-scale video classification with CNNs

- **Fovea and context streams**: the proposed multiresolution architecture aims to strike a compromise by having two separate streams of processing over two spatial resolutions (Figure 2)
- a $178 \times 178$ frame video clip forms an input to the network
- the context stream receives the downsampled frames at half the original spatial resolution ($89 \times 89$ pixels), while the fovea stream receives the center $89 \times 89$ region at the original resolution
- in this way, the total input dimensionality is halved
- notably, this design takes advantage of the camera bias present in many online videos, since the object of interest often occupies the center region

# 9.1 Large-scale video classification with CNNs

- **Architecture changes**: both streams are processed by identical networks as the full-frame models, but starting at $89 \times 89$ clips of video
- since the input is only half the spatial size of the full-frame models, the last pooling layer is taken out to ensure that both streams still terminate in a layer of size $7 \times 7 \times 256$
- the activations from both streams are concatenated and fed into the first fully-connected layer with dense connections

# 9.1 Large-scale video classification with CNNs

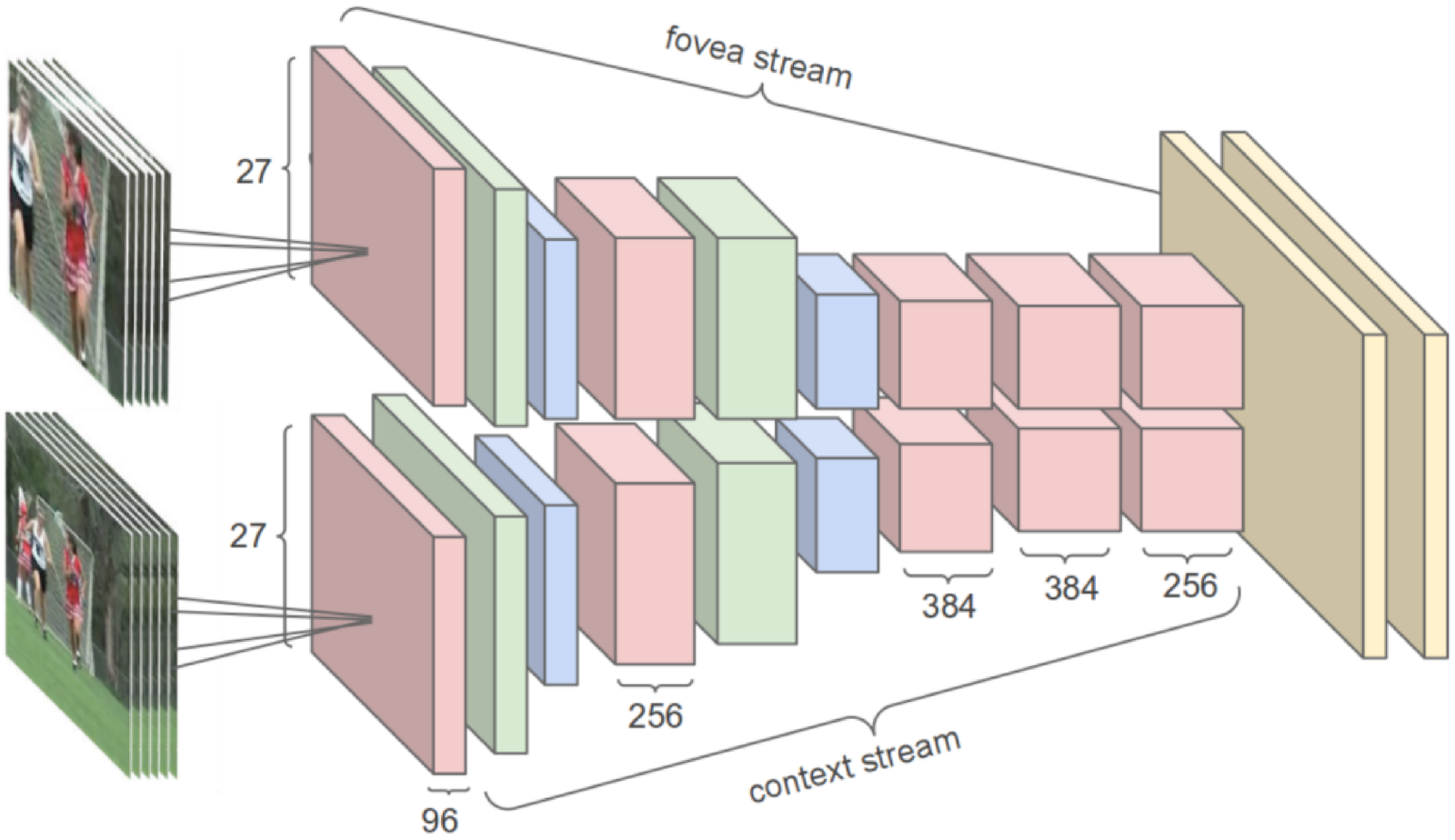



Figure 2: Multiresolution CNN architecture. Input frames are fed into two separate streams of processing: a *context stream* that models low-resolution image and a *fovea stream* that processes high-resolution center crop. Both streams consist of alternating convolution (red), normalization (green), and pooling (blue) layers. Both streams converge to two fully-connected layers (yellow).

# 9.2 Two-Stream CNNs

- video can naturally be decomposed into spatial and temporal components
- the spatial part, in the form of individual frame appearance, carries information about scenes and objects depicted in the video
- the temporal part, in the form of motion across the frames, conveys the movement of the observer (the camera) and the objects
- the proposed video recognition architecture is devised accordingly, dividing it into two streams, as shown in Figure 3
- each stream is implemented using a deep CNN, softmax scores of which are combined by late fusion
- two fusion methods are considered: averaging and training a multi-class linear SVM on stacked $\ell_2$-normalized softmax scores as features

# 9.2 Two-Stream CNNs

- **Spatial stream CNN** operates on individual video frames, effectively performing action recognition from still images
- the static appearance by itself is a useful clue, since some actions are strongly associated with particular objects
- in fact, action classification from still frames (the spatial recognition stream) is fairly competitive on its own
- since a spatial CNN is essentially an image classification architecture, the network can be pre-trained on a large image classification dataset, such as the ImageNet dataset
- next, we describe the temporal stream CNN, which exploits motion and significantly improves accuracy

# 9.2 Two-Stream CNNs

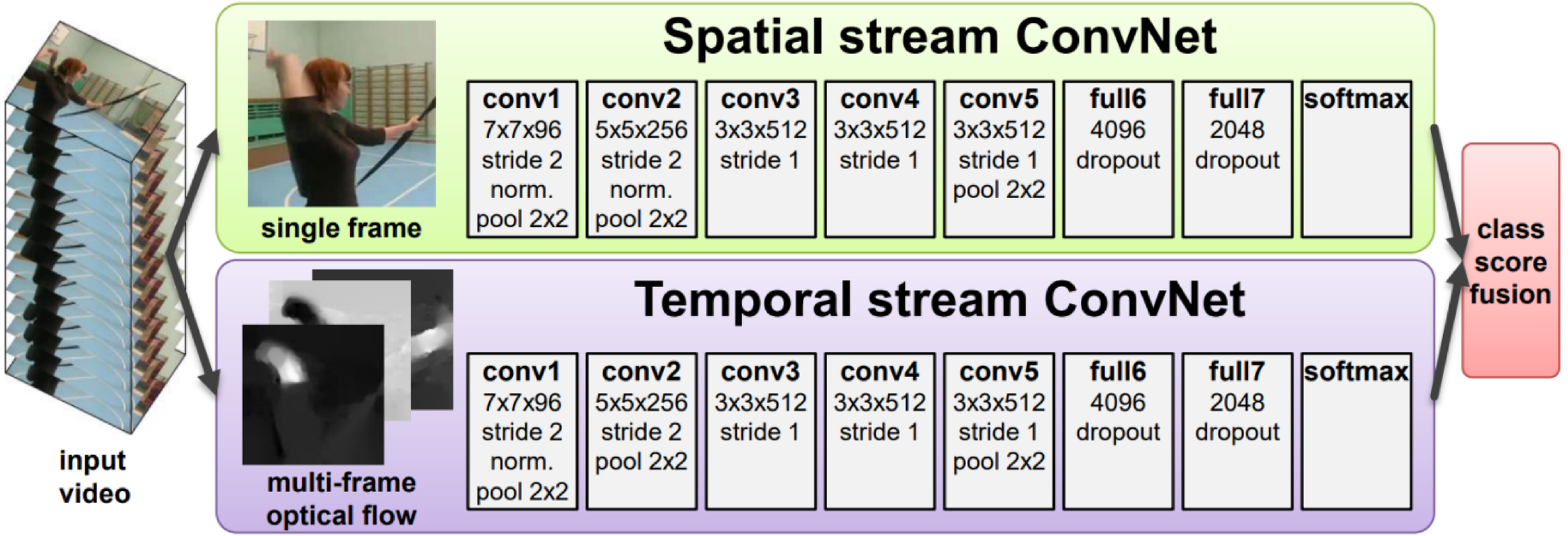



Figure 3: Two-stream architecture for video classification.

# 9.2 Two-Stream CNNs

- **Optical flow CNNs**: now, we describe a CNN model, which forms the temporal recognition stream of the architecture
- unlike the CNNs models, the input to the model is formed by stacking optical flow displacement fields between several consecutive frames
- such input explicitly describes the motion between video frames, which makes the recognition easier, as the network does not need to estimate motion implicitly
- several variations of the optical flow-based input are considered, which are described below

# 9.2 Two-Stream CNNs

- **Optical flow stacking**: a dense optical flow can be seen as a set of displacement vector fields $\boldsymbol{d}_t$ between the pairs of consecutive frames $t$ and $t+1$
- by $\boldsymbol{d}_t(u, v)$ we denote the displacement vector at the point $(u, v)$ in frame $t$, which moves the point to the corresponding point in the following frame $t+1$
- the horizontal and vertical components of the vector field, $d_t^x$ and $d_t^y$, can be seen as image channels (shown in Figure 4), well suited to recognition using a convolutional network
- to represent the motion across a sequence of frames, the flow channels $d_t^{x,y}$ of $L$ consecutive frames are stacked to form a total of $2L$ input channels
- more formally, let $w$ and $h$ be the width and height of a video; a CNN input volume $I_\tau \in \mathbb{R}^{w \times h \times 2L}$ for an arbitrary frame $\tau$ is then constructed as follows:

$$\begin{aligned} I_\tau(u, v, 2k-1) &= d^x_{\tau+k-1}(u, v), \\ I_\tau(u, v, 2k) &= d^y_{\tau+k-1}(u, v), \quad u = 1 \ldots w, \quad v = 1 \ldots h, \quad k = 1 \ldots L. \end{aligned} \tag{1}$$

- for an arbitrary point $(u, v)$, the channels $I_\tau(u, v, c)$, $c = 1 \ldots 2L$ encode the motion at that point over a sequence of $L$ frames (as illustrated in Figure 5 left)

# 9.2 Two-Stream CNNs

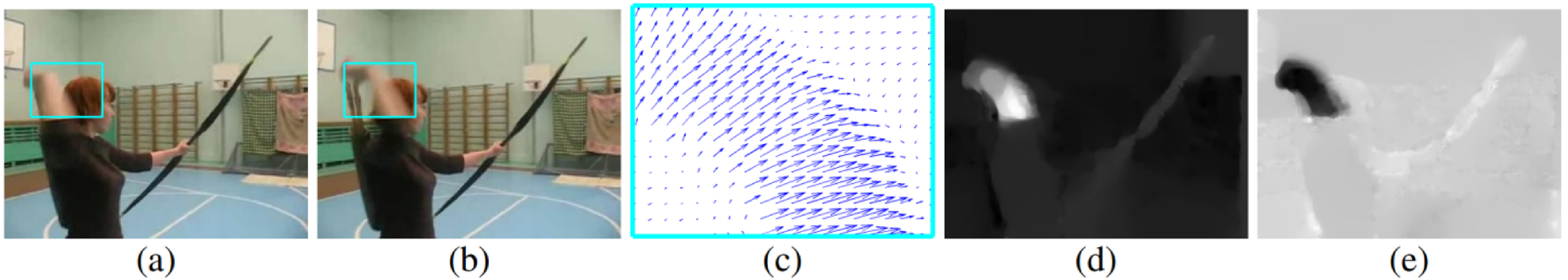



Figure 4: Optical flow. (a), (b): a pair of consecutive video frames with the area around a moving hand outlined with a cyan rectangle. (c): a close-up of dense optical flow in the outlined area; (d): horizontal component $d^x$ of the displacement vector field (higher intensity corresponds to positive values, lower intensity to negative values). (e): vertical component $d^y$. Note how (d) and (e) highlight the moving hand and bow. The input to a CNN contains multiple flows.

# 9.2 Two-Stream CNNs

- **Trajectory stacking**: an alternative motion representation replaces the optical flow, sampled at the same locations across several frames, with the flow, sampled along the motion trajectories
- in this case, the input volume $I_\tau$, corresponding to a frame $\tau$, takes the following form:

$$\begin{aligned} I_\tau(u, v, 2k-1) &= d^x_{\tau+k-1}(\boldsymbol{p}_k), \\ I_\tau(u, v, 2k) &= d^y_{\tau+k-1}(\boldsymbol{p}_k), \quad u = 1 \ldots w, \quad v = 1 \ldots h, \quad k = 1 \ldots L, \end{aligned} \tag{2}$$

where $\boldsymbol{p}_k$ is the $k$th point along the trajectory, which starts at the location $(u, v)$ in the frame $\tau$ and is defined by the following recurrence relation:

$$\boldsymbol{p}_1 = (u, v), \qquad \boldsymbol{p}_k = \boldsymbol{p}_{k-1} + \boldsymbol{d}_{\tau+k-2}(\boldsymbol{p}_{k-1}), \; k > 1.$$

- compared to the input volume representation (1), where the channels $I_\tau(u, v, c)$ store the displacement vectors at the locations $(u, v)$, the input volume (2) stores the vectors sampled at the locations $\boldsymbol{p}_k$ along the trajectory (as illustrated in Figure 5 right)

# 9.2 Two-Stream CNNs

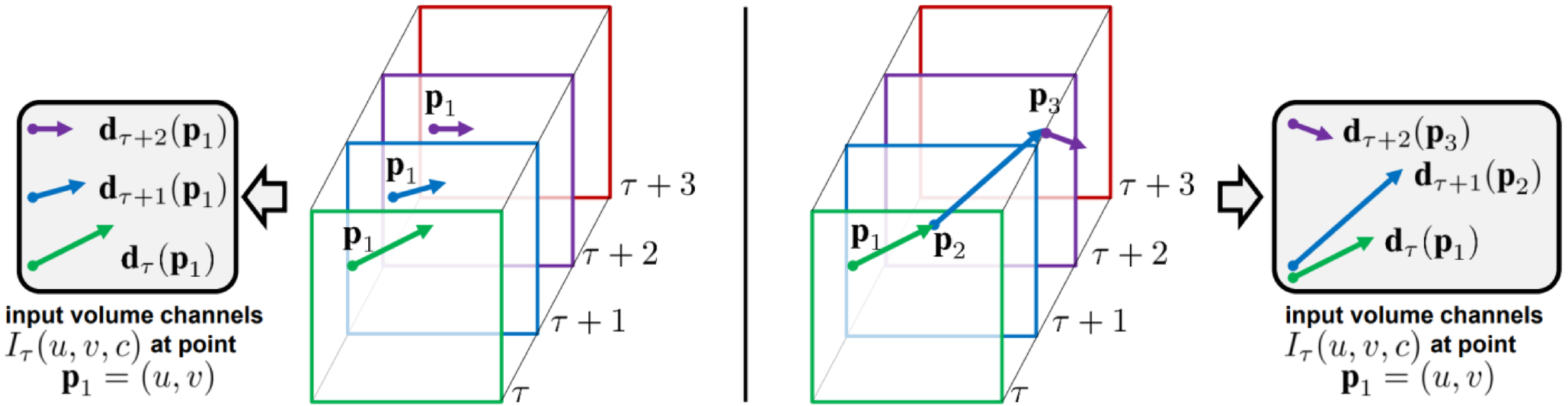



Figure 5: CNN input derivation from the multi-frame optical flow. Left: optical flow stacking (1) samples the displacement vectors ***d*** at the same location in multiple frames. Right: trajectory stacking (2) samples the vectors along the trajectory. The frames and the corresponding displacement vectors are shown with the same color.

# 9.3 3D CNNs (C3D)

- in 2D CNNs, 2D convolution is performed at the convolutional layers to extract features from local neighborhood on feature maps in the previous layer
- then an additive bias is applied and the result is passed through an activation function
- formally, the value of an unit at position $(x, y)$ in the $j$th feature map in the $i$th layer, denoted as $v_{ij}^{xy}$, is given by

$$v_{ij}^{xy} = \phi\left(b_{ij} + \sum_{m}\sum_{p=0}^{P_i-1}\sum_{q=0}^{Q_i-1} w_{ijm}^{pq} v_{(i-1)m}^{(x+p)(y+q)}\right),$$

  where $\phi$ is the activation function, $b_{ij}$ is the bias for this feature map, $m$ indexes over the set of feature maps in the $(i-1)$th layer connected to the current feature map, $w_{ijk}^{pq}$ is the value at the position $(p, q)$ of the kernel connected to the $k$th feature map, and $P_i$ and $Q_i$ are the height and width of the kernel, respectively
- in the downsampling layers, the resolution of the feature maps is reduced by pooling over local neighborhood on the feature maps in the previous layer, thereby enhancing the invariance to distortions on the inputs
- a CNN architecture can be constructed by stacking multiple layers of convolution and downsampling in an alternating fashion

# 9.3 3D CNNs (C3D)

- in 2D CNNs, convolutions are applied on the 2D feature maps to compute features from the spatial dimensions only
- when applied to video analysis problems, it is desirable to capture the motion information encoded in multiple contiguous frames
- to this end, 3D convolutions can be used in the convolution stages of CNNs to compute features from both spatial and temporal dimensions
- the 3D convolution is achieved by convolving a 3D kernel to the cube formed by stacking multiple contiguous frames together
- by this construction, the feature maps in the convolution layer are connected to multiple contiguous frames in the previous layer, thereby capturing motion information
- formally, the value at position $(x, y, z)$ on the $j$th feature map in the $i$th layer is given by

$$v_{ij}^{xyz} = \phi\left(b_{ij} + \sum_{m}\sum_{p=0}^{P_i-1}\sum_{q=0}^{Q_i-1}\sum_{r=0}^{R_i-1} w_{ijm}^{pqr}\, v_{(i-1)m}^{(x+p)(y+q)(z+r)}\right),$$

  where $R_i$ is the size of the 3D kernel along the temporal dimension, $w_{ijm}^{pqr}$ is the $(p, q, r)$th value of the kernel connected to the $m$th feature map in the previous layer
- a comparison of 2D and 3D convolutions is given in Figure 6

# 9.3 3D CNNs (C3D)

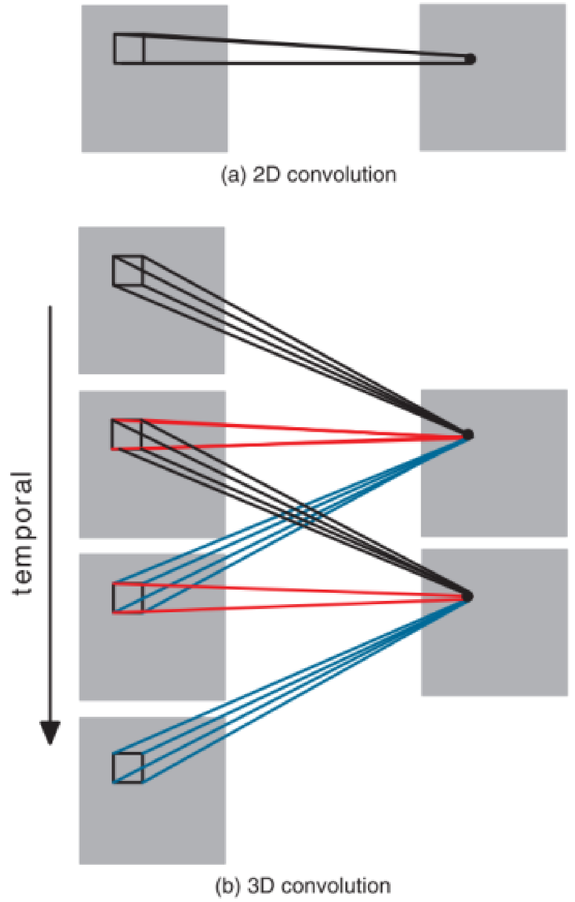



Figure 6: Comparison of (a) 2D and (b) 3D convolutions. In (b) the size of the convolution kernel in the temporal dimension is 3 and the sets of connections are color-coded so that the shared weights are in the same color. In 3D convolution, the same 3D kernel is applied to overlapping 3D cubes in the input video to extract motion features.

# 9.3 3D CNNs (C3D)

- note that a 3D convolutional kernel can only extract one type of features from the frame cube, since the kernel weights are replicated across the entire cube
- a general design principle of CNNs is that the number of feature maps should be increased in late layers by generating multiple types of features from the same set of lower-level feature maps
- similarly to the case of 2D convolution, this can be achieved by applying multiple 3D convolutions with distinct kernels to the same location in the previous layer (Figure 7)

# 9.3 3D CNNs (C3D)

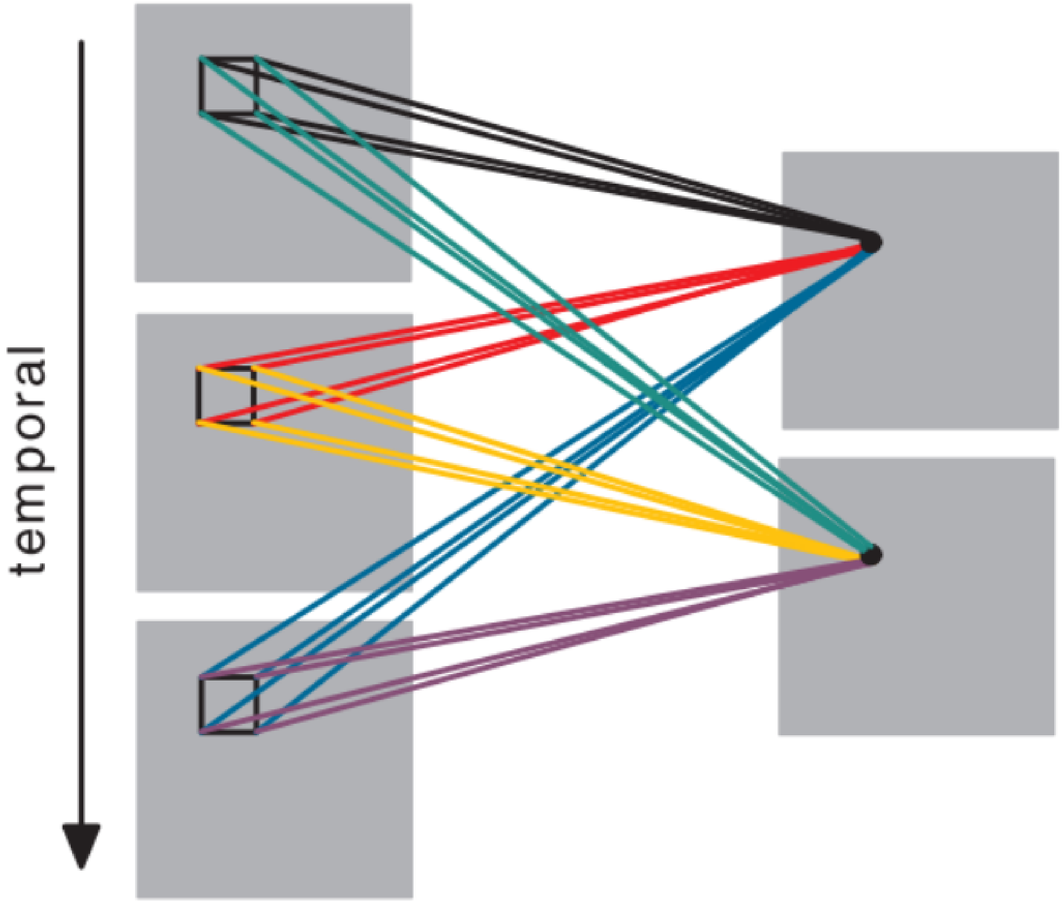



Figure 7: Extraction of multiple features from contiguous frames. Multiple 3D convolutions can be applied to contiguous frames to extract multiple features. As in Figure 6, the sets of connections are color-coded so that the shared weights are in the same color. Note that all six sets of connections do not share weights, resulting in two different feature maps on the right.

# 9.3 3D CNNs (C3D)

- now, we explain in detail the basic operations of 3D CNNs and analyze different architectures for 3D CNNs
- 3D CNNs are believed to be well-suited for spatio-temporal feature learning
- compared to 2D CNNs, 3D CNNs have the ability to model temporal information better, owing to 3D convolution and 3D pooling operations
- in 3D CNNs, convolution and pooling operations are performed spatio-temporally, while in 2D CNNs they are done only spatially
- Figure 6 illustrates the difference: 2D convolution applied on an image will output an image, 2D convolution applied on multiple images (treating them as different channels) also results in an image
- hence, 2D CNNs lose temporal information of the input signal right after every convolution operation
- only 3D convolution preserves the temporal information of the input signals, resulting in an output volume
- the same phenomena is applicable for 2D and 3D polling

# 9.3 3D CNNs (C3D)

- now, we try to identify a good architecture for 3D CNNs
- according to the findings in 2D CNNs, small receptive fields of $3 \times 3$ convolution kernels with deeper architectures yield best results
- hence, for the architecture search study, the spatial receptive field is fixed to $3 \times 3$, and only the temporal depth of the 3D convolution kernels is varied
- **Notations**: for simplicity, from now on, we refer to video clips with a size of $c \times l \times h \times w$, where $c$ is the number of channels, $l$ is length in number of frames, $h$ and $w$ are the height and width of the frame, respectively
- we also refer to 3D convolution and pooling kernel size by $d \times k \times k$, where $d$ is the kernel temporal depth and $k$ is the kernel spatial size

# 9.3 3D CNNs (C3D)

- **Common network settings**: we now describe the network settings that are common to all the networks
- the networks are set up to take video clips as inputs and predict the class labels
- all video frames are resized into $128 \times 171$
- videos are split into non-overlapped 16-frame clips, which are then used as input to the networks
- the input dimensions are $3 \times 16 \times 128 \times 171$
- the networks have 5 convolution layers and 5 pooling layers (each convolution layer is immediately followed by a pooling layer), 2 fully-connected layers, and a softmax loss layer to predict action labels

# 9.3 3D CNNs (C3D)

- the number of filters for the 5 convolution layers from 1 to 5 are 64, 128, 256, 256, 256, respectively
- all convolution kernels have a size of $d$, where $d$ is the kernel temporal depth (we will later vary the value $d$ of these layers to search for a good 3D architecture)
- all of these convolution layers are applied with appropriate padding (both spatial and temporal) and stride 1, thus there is no change in terms of size from the input to the output of these convolution layers
- all pooling layers are max-pooling with kernel size $2 \times 2 \times 2$ (except for the first layer) with stride 1, which means the size of output signal is reduced by a factor of 8 compared with the input signal
- the first pooling layer has kernel size $1 \times 2 \times 2$ with the intention of not to merge the temporal signal too early and also to satisfy the clip length of 16 frames (e.g., we can temporally pool with factor 2 at most 4 times before completely collapsing the temporal signal)
- the two fully-connected layers have 2048 outputs

# 9.3 3D CNNs (C3D)

- **Varying network architectures**: we are mainly interested in how to aggregate temporal information through the deep networks
- to search for a good 3D CNN architecture, only the kernel temporal depth $d_i$ of the convolution layers is varied, while keeping all other common settings fixed as stated above
- two types of architectures are experimented with: 1) homogeneous temporal depth: all convolution layers have the same kernel temporal depth; and 2) varying temporal depth: kernel temporal depth is changing across the layers
- for homogeneous setting, 4 networks having kernel temporal depth of $d$ equal to 1, 3, 5, and 7 are experimented with
- these networks are named as *depth-d*, where $d$ is their homogeneous temporal depth
- note that *depth-1* net is equivalent to applying 2D convolutions on separate frames
- for the varying temporal depth setting, two networks with temporal depth increasing: 3-3-5-5-7 and decreasing: 7-5-5-3-3 from the first to the fifth convolution layer, respectively, are experimented with

# 9.3 3D CNNs (C3D)

- note that all of these networks have the same size of the output signal at the last pooling layer, thus they have the same number of parameters for the fully-connected layers
- their number of parameters is only different at convolution layers due to different kernel temporal depth
- these differences are quite minor compared to millions of parameters in the fully-connected layers
- for example, any two of the above networks with temporal depth difference of 2, only have 17K parameters fewer or more from each other
- the biggest difference in number of parameters is between *depth-1* network and *depth-7* network, where *depth-7* network has 51K more parameters, which is less than 0.3% of the total of 17.5 million parameters of each network
- this indicates that the learning capacities of the networks are comparable and the differences in number of parameters should not affect the results of the architecture search

# 9.3 3D CNNs (C3D)

- it is found that *depth-3* performs best among the homogeneous networks
- note that *depth-1* is significantly worse than the other networks, which is believed to be due to lack of motion modeling
- compared to the varying temporal depth networks, *depth-3* is the best performer, but the gap is smaller
- this suggests $3 \times 3 \times 3$ is the best kernel choice for 3D CNNs and 3D CNNs are consistently better than 2D CNNs for video classification

# 9.3 3D CNNs (C3D)

- **Network architecture**: the architecture search findings indicate that the homogeneous setting with convolution kernels of $3 \times 3 \times 3$ is the best option for 3D CNNs
- this finding is also consistent with a similar finding in 2D CNNs
- with a large-scale dataset, one can train a 3D CNN with $3 \times 3 \times 3$ kernel as deep as possible, subject to the machine memory limit and computation affordability
- the proposed 3D CNN is designed to have 8 convolution layers, 5 pooling layers, followed by two fully-connected layers, and a softmax output layer
- the network architecture is presented in Figure 8
- this network is called C3D
- all of 3D convolution filters are $3 \times 3 \times 3$ with stride $1 \times 1 \times 1$
- all 3D pooling layers are $2 \times 2 \times 2$ with stride $2 \times 2 \times 2$, except for `pool1`, which has kernel size of $1 \times 2 \times 2$ and stride $1 \times 2 \times 2$, with the intention of preserving the temporal information in the early phase
- each fully-connected layer has 4096 output units

# 9.3 3D CNNs (C3D)

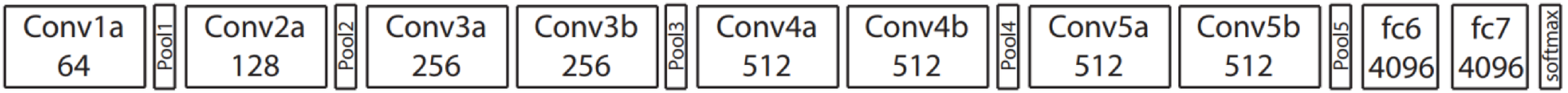



Figure 8: C3D architecture. C3D net has 8 convolution, 5 max-pooling, and 2 fully-connected layers, followed by a softmax output layer. All 3D convolution kernels are $3 \times 3 \times 3$ with stride 1 in both spatial and temporal dimensions. Number of filters are denoted in each box. The 3D pooling layers are denoted from `pool1` to `pool5`. All pooling kernels are $2 \times 2 \times 2$, except for `pool1`, which is $1 \times 2 \times 2$. Each fully-connected layer has 4096 output units.

# 9.4 Temporal Segment Networks (TSN)

- an obvious problem of the two-stream CNNs in their previous forms is their inability in modeling long-range temporal structure
- this is mainly due to their limited access to temporal context, as they are designed to operate only on a single frame (spatial networks) or a single stack of frames in a short snippet (temporal network)
- however, complex actions, such as sports actions, comprise multiple stages spanning over a relatively long time
- it would be quite a loss failing to utilize long-range temporal structures in these actions into CNN training
- to tackle this issue, temporal segment network is proposed, a video-level framework as shown in Figure 9, to enable to model dynamics throughout the whole video

# 9.4 Temporal Segment Networks (TSN)

- specifically, the proposed temporal segment network framework, aiming to utilize the visual information of entire videos to perform video-level prediction, is also composed of spatial stream CNNs and temporal stream CNNs
- instead of working on single frames or frame stacks, temporal segment networks operate on a sequence of short snippets, sparsely sampled from the entire video
- each snippet in this sequence will produce its own preliminary prediction of the action classes
- then, a consensus among the snippets will be derived as the video-level prediction
- in the learning process, the loss values of video-level predictions, other than those of snippet-level predictions, which were used in two-stream CNNs, are optimized by iteratively updating the model parameters

# 9.4 Temporal Segment Networks (TSN)

- formally, given a video $V$, we divide it into $K$ segments $\{S_1, S_2, \ldots, S_K\}$ of equal durations
- then, the temporal segment network models a sequence of snippets as follows:

$$TSN(T_1, T_2, \ldots, T_K) = \mathcal{H}(\mathcal{G}(\mathcal{F}(T_1; \boldsymbol{W}), F(T_2; \boldsymbol{W}), \ldots, F(T_K; \boldsymbol{W}))).$$

- here $(T_1, T_2, \ldots, T_K)$ is a sequence of snippets
- each snippet $T_k$ is randomly sampled from its corresponding segment $S_k$
- $\mathcal{F}(T_k; \boldsymbol{W})$ is the function representing a CNN with parameters $\boldsymbol{W}$, which operates on the short snippet $T_k$ and produces class scores for all the classes
- the segmental consensus function $\mathcal{G}$ combines the outputs from multiple short snippets to obtain a consensus of class hypothesis among them
- based on this consensus, the prediction function $\mathcal{H}$ predicts the probability of each action class for the whole video
- here, the widely used softmax function for $\mathcal{H}$ is chosen

# 9.4 Temporal Segment Networks (TSN)

- combining with standard categorical cross-entropy loss, the final loss function regarding the segmental consensus $\boldsymbol{G} = \mathcal{G}(\mathcal{F}(T_1; \boldsymbol{W}), F(T_2; \boldsymbol{W}), \ldots, F(T_K; \boldsymbol{W}))$ is formed as

$$\mathcal{L}(y, \boldsymbol{G}) = -\sum_{i=1}^{C} y_i \left( G_i - \log \left( \sum_{j=1}^{C} \exp G_j \right) \right),$$

  where $C$ is the number of action classes and $y_i$ the ground truth label concerning class $i$
- the number of snippets $K$ is set to 3
- the form of consensus function $\mathcal{G}$ remains an open question
- in the paper, the simplest form of $\mathcal{G}$ is used, where $G_i = g(\mathcal{F}_i(T_1), \ldots, \mathcal{F}_i(T_K))$
- here, a class score $G_i$ is inferred from the scores of the same class on all the snippets, using an aggregation function $g$
- several different forms of the aggregation function $g$, including evenly averaging, maximum, and weighted averaging were empirically evaluated
- among them, evenly averaging had the best performance

# 9.4 Temporal Segment Networks (TSN)

- this temporal segment network is differentiable or at least has subgradients, depending on the choice of $g$
- this allows utilizing the multiple snippets to jointly optimize the model parameters $\boldsymbol{W}$ with standard backpropagation algorithms
- in the backpropagation process, the gradients of model parameters $\boldsymbol{W}$ with respect to the loss value $\mathcal{L}$ can be derived as

$$\frac{\partial \mathcal{L}(y, \boldsymbol{G})}{\partial \boldsymbol{W}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{G}} \sum_{k=1}^{K} \frac{\partial \mathcal{G}}{\partial \mathcal{F}(T_k)} \frac{\partial \mathcal{F}(T_k)}{\partial \boldsymbol{W}}, \tag{3}$$

where $K$ is the number of segments the temporal segment network uses

# 9.4 Temporal Segment Networks (TSN)

- when a gradient-based optimization method, like stochastic gradient descent (SGD), is used to learn the model parameters, relation (3) guarantees that the parameter updates are utilizing the segmental consensus $\boldsymbol{G}$ derived from all snippet-level prediction
- optimized in this manner, the temporal segment network can learn model parameters from the entire video, rather than a short snippet
- meanwhile, by fixing $K$ for all videos, a sparse temporal sampling strategy is assembled, where the sampled snippets contain only a small portion of the frames
- this drastically reduces the computational cost for evaluating CNNs on the frames, compared with previous works using densely sampled frames

# 9.4 Temporal Segment Networks (TSN)

- temporal segment network provides a solid framework to perform video-level learning, but, to achieve optimal performance, a few practical concerns have to be taken care of, for example the limited number of training samples
- to this end, we study a series of good practices in training deep CNNs on video data, which are also directly applicable in learning temporal segment networks

# 9.4 Temporal Segment Networks (TSN)

- **Network architecture** is an important factor in neural network design
- several works have shown that deeper structures improve object recognition performance
- however, the original two-stream CNNs employed a relatively shallow network structure
- in the paper, the Inception with Batch Normalization (BN-Inception) is chosen as building block, due to its good balance between accuracy and efficiency
- the original BN-Inception architecture is adapted to the design of two-stream CNNs
- like in the original two-stream CNNs, the spatial stream CNN operates on a single RGB image, and the temporal stream CNN takes a stack of consecutive optical flow fields as input

# 9.4 Temporal Segment Networks (TSN)

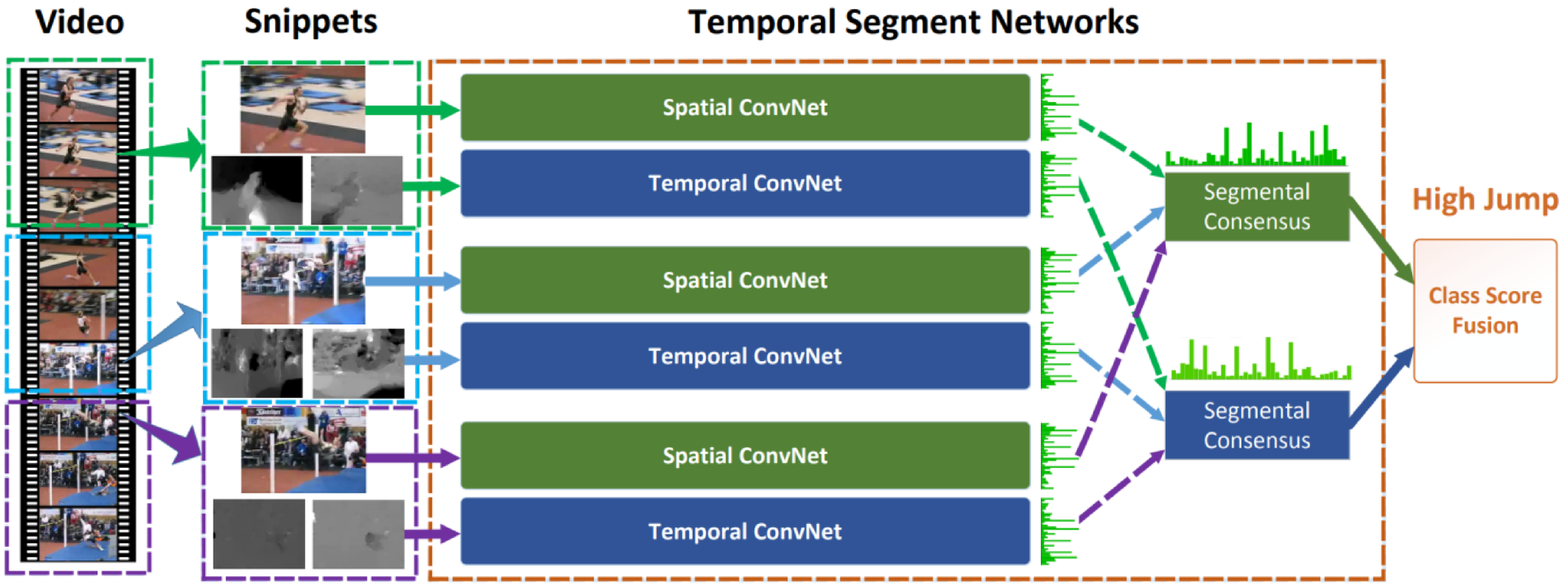



Figure 9: Temporal segment network: one input video is divided into $K$ segments and a short snippet is randomly selected from each segment. The class scores of different snippets are fused by the segmental consensus function to yield segmental consensus, which is a video-level prediction. Predictions from all modalities are then fused to produce the final prediction. CNNs on all snippets share parameters.

# 9.4 Temporal Segment Networks (TSN)

- **Network inputs**: we are also interested in exploring more input modalities to enhance the discriminative power of temporal segment networks
- originally, the two-stream CNNs used RGB images for the spatial stream and stacked optical flow fields for the temporal stream
- here, two extra modalities are proposed, namely *RGB difference* and *warped optical flow fields*

# 9.4 Temporal Segment Networks (TSN)

- a single RGB image usually encodes static appearance at a specific time point and lacks the contextual information about previous and next frames
- as shown in Figure 10, RGB difference between two consecutive frames describes the appearance change, which may correspond to the motion salient region
- thus, adding stacked RGB difference as another input modality is experimented with, and its performance in action recognition is investigated

# 9.4 Temporal Segment Networks (TSN)

- the temporal stream CNNs take optical flow field as input and aim to capture the motion information
- in realistic videos, however, there usually exists camera motion, and optical flow fields may not concentrate on the human action
- as shown in Figure 10, a remarkable amount of horizontal movement is highlighted in the background due to the camera motion
- warped optical flow fields are proposed as an additional input modality
- the warped optical flow is extracted by first estimating the homography matrix and then compensating camera motion
- as shown in Figure 10, the warped optical flow suppresses the background motion and makes motion concentrate on the actor

# 9.4 Temporal Segment Networks (TSN)

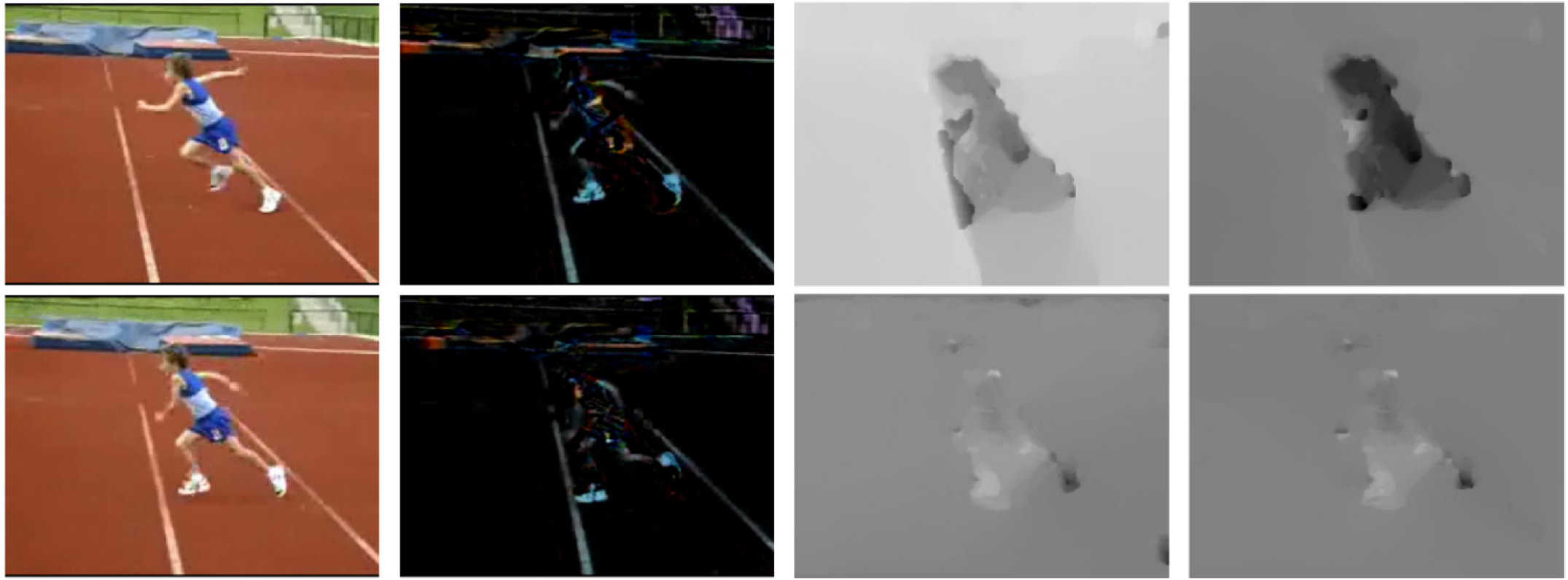


Figure 10: Examples of four types of input modality: RGB images, RGB difference, optical flow fields ($x$, $y$ directions), and warped optical flow fields ($x$, $y$ directions).

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

- some of the major differences in previous video architectures are whether the convolutional and layers operators use 2D (image-based) or 3D (video-based) kernels; whether the input to the network is just an RGB video or it also includes pre-computed optical flow; and, in the case of 2D CNNs, how information is propagated across frames, which can be done either using temporally-recurrent layers such as LSTMs, or feature aggregation over time
- this paper compares and studies a subset of models that span most of this space
- among 2D CNNs methods, CNNs with LSTMs on top are considered, and two-stream networks with two different types of stream fusion
- a 3D CNN, C3D, is also considered

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

- as the main technical contribution, the paper introduces Two-Stream Inflated 3D CNNs (I3D)
- due to the high-dimensionality of their parameterization, and the lack of labeled video data, previous 3D CNNs have been relatively shallow (up to 8 layers)
- the paper makes the observation that very deep image classification networks, such as Inception, VGG-16, and ResNet, can be trivially inflated into spatio-temporal feature extractors, and that their pre-trained weights provide a valuable initialization
- a two-stream configuration is also found to still be useful

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

- a graphical overview of the five types of architectures that are evaluated is shown in Figure 11
- many of these models (all but C3D) have an ImageNet pre-trained model as a subcomponent
- the experimental strategy assumes a common ImageNet pre-trained image classification architecture as backbone, and, for this, Inception-V1 with batch normalization is chosen, which is morphed in different ways
- the expectation is that, with this backbone in common, we will be able to tease apart those changes that benefit action classification the most

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

- **CNN+LSTM**: the high performance of image classification networks makes it appealing to try to reuse them with as minimal change as possible for video
- this can be achieved by using them to extract features independently from each frame, then pooling their predictions across the whole video
- but while convenient in practice, it has the issue of entirely ignoring temporal structure (e.g., models can't potentially distinguish opening from closing a door)

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

- in theory, a more satisfying approach is to add a recurrent layer to the model, such as an LSTM, which can encode state and capture temporal ordering and long-range dependencies
- an LSTM layer with batch normalization is positioned after the last average pooling layer of Inception-V1, with 512 hidden units
- a fully-connected layer is added on top for the classifier
- the model is trained using cross-entropy losses on the outputs at all time steps
- during testing, only the output on the last frame is considered
- input video frames are subsampled by keeping one out of every 5, from an original 25 frames-per-second stream

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

- **3D CNNs**: 3D CNNs seem like a natural approach to video modeling, and are just like standard convolutional networks, but with spatio-temporal filters
- they have a very important characteristic: they directly create hierarchical representations of spatio-temporal data
- one issue with these models is that they have many more parameters than 2D CNNs because of the additional kernel dimension, and this makes them harder to train
- also, they seem to preclude the benefits of ImageNet pre-training, and consequently previous work has defined relatively shallow custom architectures and trained them from scratch
- results on benchmarks have shown promise, but have not been competitive with state-of-the-art

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

- for the paper, a small variation of C3D was implemented, which has 8 convolutional layers, 5 pooling layers, and 2 fully-connected layers at the top
- the inputs to the model are short 16-frame clips with $112 \times 112$-pixel crops, as in the original implementation
- differently from the original paper, batch normalization is used after all convolutional and fully-connected layers
- another difference to the original model is in the first pooling layer, a temporal stride of 2 is used instead of 1, which reduces the memory footprint and allows for bigger batches – this was important for batch normalization (especially after the fully-connected layers, where there is no weight tying)

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

- **Two-Stream Networks**: LSTMs on features from the last layers of CNNs can model high-level variation, but may not be able to capture fine low-level motion, which is critical in many cases
- it is also expensive to train, as it requires unrolling the network through multiple frames for backpropagation-through-time
- a different, very practical approach, models short temporal snapshots of videos by averaging the predictions from a single RGB frame and a stack of 10 externally computed optical flow frames, after passing them through two replicas of an ImageNet pre-trained CNN
- the flow stream has an adapted input convolutional layer with twice as many input channels as flow frames (because flow has two channels, horizontal and vertical), and at test time multiple snapshots are sampled from the video and the action prediction is averaged
- this was shown to get very high performance on existing benchmarks, while being very efficient to train and test

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

- an extension to this architecture fuses the spatial and flow streams after the last network convolutional layer, showing some improvement in performance, while requiring less test-time augmentation (snapshot sampling)
- the current implementation follows this model approximately, using Inception-V1
- the inputs to the network are 5 consecutive RGB frames sampled 10 frames apart, as well as the corresponding optical flow snippets
- the spatial and motion features before the last average pooling layer of Inception-V1 ($5 \times 7 \times 7$ feature grids, corresponding to time, $x$ and $y$ dimensions) are passed through a $3 \times 3 \times 3$ 3D convolutional layer with 512 output channels, followed by a $3 \times 3 \times 3$ 3D max-pooling layer and through a final fully-connected layer
- the weights of these new layers are randomly initialized
- both models, the original two-stream and the 3D fused version, are trained end-to-end (including the two-stream averaging process in the original model)

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

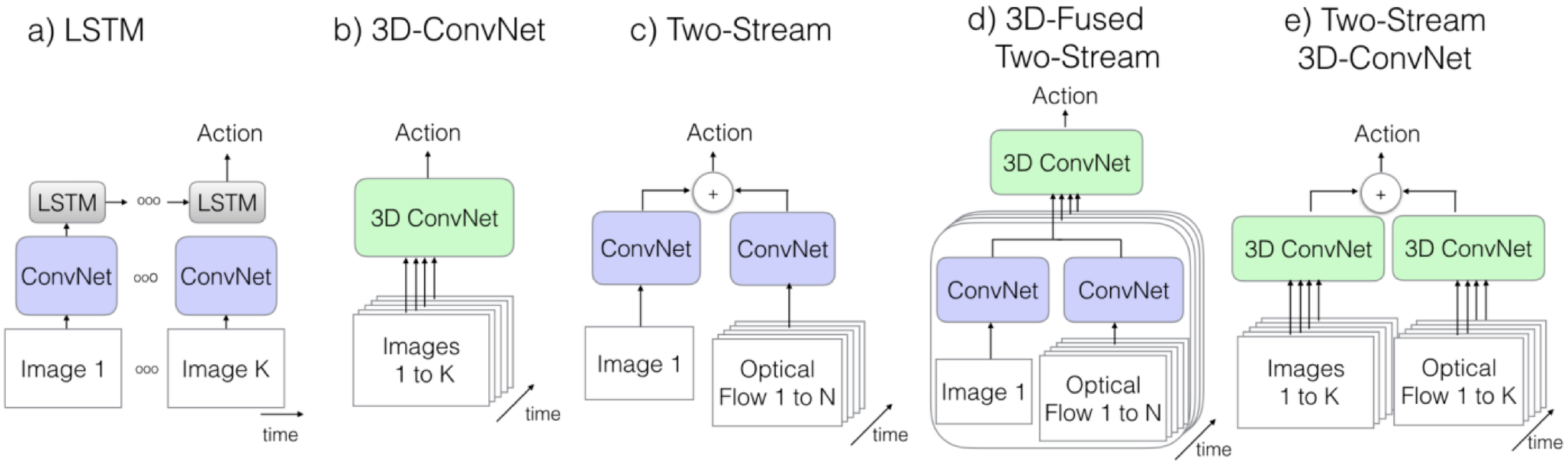



Figure 11: Video architectures considered in the paper. *K* stands for the total number of frames in a video, whereas *N* stands for a subset of neighboring frames of the video.

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

- **Two-Stream Inflated 3D CNNs**: with this architecture, it is shown how 3D CNNs can benefit from ImageNet 2D CNN designs and, optionally, from their learned parameters
- a two-stream configuration is also adopted here – it can be shown that, while 3D CNNs can directly learn about temporal patterns from an RGB stream, their performance can still be greatly improved by including an optical flow stream

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

- **Inflating 2D CNNs into 3D**: a number of very successful image classification architectures have been developed over the years, in part through painstaking trial and error
- instead of repeating the process for spatio-temporal models, the paper proposes to simply convert successful image (2D) classification models into 3D CNNs
- this can be done by starting with a 2D architecture, and inflating all the filters and pooling kernels – endowing them with an additional temporal dimension
- filters are typically square and we just make them cubic – $N \times N$ filters become $N \times N \times N$

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

- **Bootstrapping 3D filters from 2D Filters**: besides the architecture, one may also want to bootstrap parameters from the pre-trained ImageNet models
- to do this, we observe that an image can be converted into a (boring) video by copying it repeatedly into a video sequence
- the 3D models can then be implicitly pre-trained on ImageNet, by satisfying what is called the boring-video fixed point: the pooled activations on a boring video should be the same as on the original single-image input
- this can be achieved, thanks to linearity, by repeating the weights of the 2D filters $N$ times along the time dimension, and rescaling them by dividing by $N$
- this ensures that the convolutional filter response is the same
- since the outputs of convolutional layers for boring videos are constant in time, the outputs of pointwise nonlinearity layers and average and max-pooling layers are the same as for the 2D case, and hence the overall network response respects the boring-video fixed point

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

- **Pacing receptive field growth in space, time, and network depth**: the boring video fixed-point leaves ample freedom on how to inflate pooling operators along the time dimension and on how to set convolutional/pooling temporal stride – these are the primary factors that shape the size of feature receptive fields
- virtually all image models treat the two spatial dimensions (horizontal and vertical) equally – pooling kernels and strides are the same
- this is quite natural and means that features deeper in the networks are equally affected by image locations increasingly far away in both dimensions
- a symmetric receptive field is however not necessarily optimal when also considering time – this should depend on frame rate and image dimensions
- if it grows too quickly in time relative to space, it may conflate edges from different objects, breaking early feature detection, while if it grows too slowly, it may not capture scene dynamics well

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

- in Inception-V1, the first convolutional layer has stride 2, then there are four max-pooling layers with stride 2 and a $7 \times 7$ average-pooling layer preceding the last linear classification layer, besides the max-pooling layers in parallel Inception branches
- the input videos are processed at 25 frames per second; it was found helpful to not perform temporal pooling in the first two max-pooling layers (by using $1 \times 3 \times 3$ kernels and stride 1 in time), while having symmetric kernels and strides in all other max-pooling layers
- the final average pooling layer uses a $2 \times 7 \times 7$ kernel
- the overall architecture is shown in Figure 12
- the model is trained using 64-frame snippets and tested using the whole videos, averaging predictions temporally

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

- **Two 3D Streams**: while a 3D CNN should be able to learn motion features from RGB inputs directly, it still performs pure feedforward computation, whereas optical flow algorithms are in some sense recurrent (e.g., they perform iterative optimization for the flow fields)
- perhaps because of this lack of recurrence, experimentally it is still found valuable to have a two-stream configuration – shown in Figure 11 e) – with one I3D network trained on RGB inputs, and another on flow inputs which carry optimized, smooth flow information
- the two networks are trained separately and their predictions are averaged at test time

# 9.5 Two-Stream Inflated 3D CNNs (I3D)

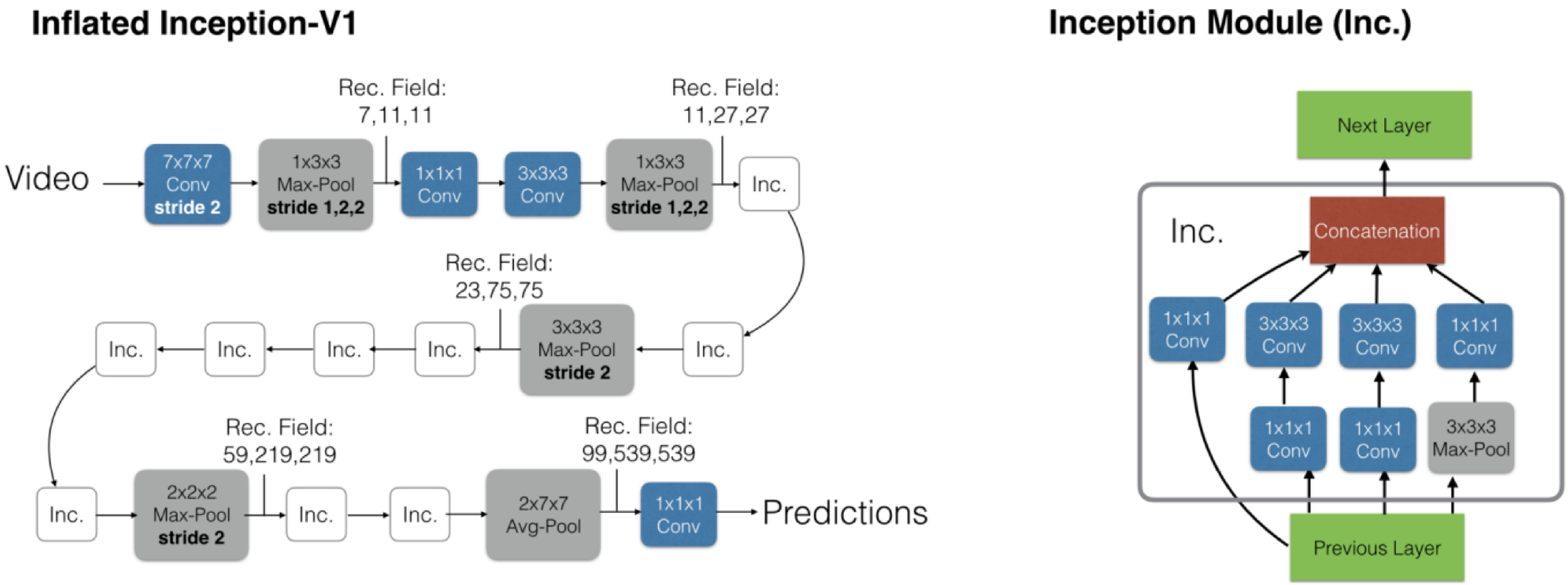



Figure 12: The Inflated Inception-V1 architecture (left) and its detailed inception submodule (right). The strides of convolution and pooling operators are 1 where not specified, and batch normalization layers, ReLUs, and the softmax at the end are not shown. The theoretical sizes of receptive field sizes for a few layers in the network are provided in the format "time,x,y" – the units are frames and pixels. The predictions are obtained convolutionally in time and averaged.

# 9.6 SlowFast Networks

- SlowFast networks can be described as a single stream architecture that operates at two different framerates, but the concept of pathways is used to reflect analogy with the biological Parvo- and Magnocellular counterparts
- the generic architecture has a Slow pathway and a Fast pathway, which are fused by lateral connections to a SlowFast network
- Figure 13 illustrates the concept

# 9.6 SlowFast Networks

- **Slow pathway**: the Slow pathway can be any convolutional model that works on a clip of video as a spatio-temporal volume
- the key concept in the Slow pathway is a large temporal stride $\tau$ on input frames, i.e., it processes only one out of $\tau$ frames
- a typical value of $\tau$ that was studied is 16—this refreshing speed is roughly 2 frames sampled per second for 30-fps videos
- denoting the number of frames sampled by the Slow pathway as $T$, the raw clip length is $T \times \tau$ frames

# 9.6 SlowFast Networks

- **Fast pathway**: in parallel to the Slow pathway, the Fast pathway is another convolutional model with the following properties
- **High frame rate**: the goal here is to have a fine representation along the temporal dimension
- the Fast pathway works with a small temporal stride of $\tau/\alpha$, where $\alpha > 1$ is the frame rate ratio between the Fast and Slow pathways
- the two pathways operate on the same raw clip, so the Fast pathway samples $\alpha T$ frames, $\alpha$ times denser than the Slow pathway
- a typical value is $\alpha = 8$ in the experiments
- the presence of $\alpha$ is in the key of the SlowFast concept (Figure 13, time axis)
- it explicitly indicates that the two pathways work on different temporal speeds, and thus drives the expertise of the two subnets instantiating the two pathways

# 9.6 SlowFast Networks

- **High temporal resolution features**: the Fast pathway not only has a high input resolution, but also pursues high-resolution features throughout the network hierarchy
- in the instantiations, no temporal downsampling layers are used (neither temporal pooling nor time-strided convolutions) throughout the Fast pathway, until the global pooling layer before classification
- as such, the feature tensors always have $\alpha T$ frames along the temporal dimension, maintaining temporal fidelity as much as possible

# 9.6 SlowFast Networks

- **Low channel capacity**: the Fast pathway also distinguishes with existing models in that it can use significantly lower channel capacity to achieve good accuracy for the SlowFast model; this makes it lightweight
- in a nutshell, the Fast pathway is a convolutional network analogous to the Slow pathway, but has a ratio of $\beta$ ($\beta < 1$) channels of the Slow pathway
- the typical value is $\beta = 1/8$ in the experiments
- notice that the computation (floating-point operations, or FLOPs) of a common layer is often quadratic in terms of its channel scaling ratio
- this is what makes the Fast pathway more computation-effective than the Slow pathway
- in the instantiations, the Fast pathway typically takes $\sim$ 20% of the total computation
- interestingly, evidence suggests that $\sim$ 15-20% of the retinal cells in the primate visual system are M-cells (that are sensitive to fast motion, but not color or spatial detail)

# 9.6 SlowFast Networks

- the low channel capacity can also be interpreted as a weaker ability of representing spatial semantics
- technically, the Fast pathway has no special treatment on the spatial dimension, so its spatial modeling capacity should be lower than the Slow pathway because of fewer channels
- the good results of the model suggest that it is a desired tradeoff for the Fast pathway to weaken its spatial modeling ability, while strengthening its temporal modeling ability
- motivated by this interpretation, different ways of weakening spatial capacity in the Fast pathway are also explored, including reducing input spatial resolution and removing color information
- as is shown by experiments, these versions can all give good accuracy, suggesting that a lightweight Fast pathway with less spatial capacity can be made beneficial

# 9.6 SlowFast Networks

- **Lateral connections**: the information of the two pathways is fused, so one pathway is not unaware of the representation learned by the other pathway
- this is implemented by lateral connections, which have been used to fuse optical flow-based, two-stream networks
- in image object detection, lateral connections are a popular technique for merging different levels of spatial resolution and semantics
- one lateral connection is attached between the two pathways for every “stage" (Figure 13)
- specifically for ResNets, these connections are right after $\text{pool}_1$, $\text{res}_2$, $\text{res}_3$, and $\text{res}_4$

# 9.6 SlowFast Networks

- the two pathways have different temporal dimensions, so the lateral connections perform a transformation to match them
- unidirectional connections that fuse features of the Fast pathway into the Slow one (Figure 13) are used
- bidirectional fusion has been experimented with and similar results have been found
- finally, a global average pooling is performed on each pathway's output
- then, two pooled feature vectors are concatenated as the input to the fully-connected classifier layer

# 9.6 SlowFast Networks

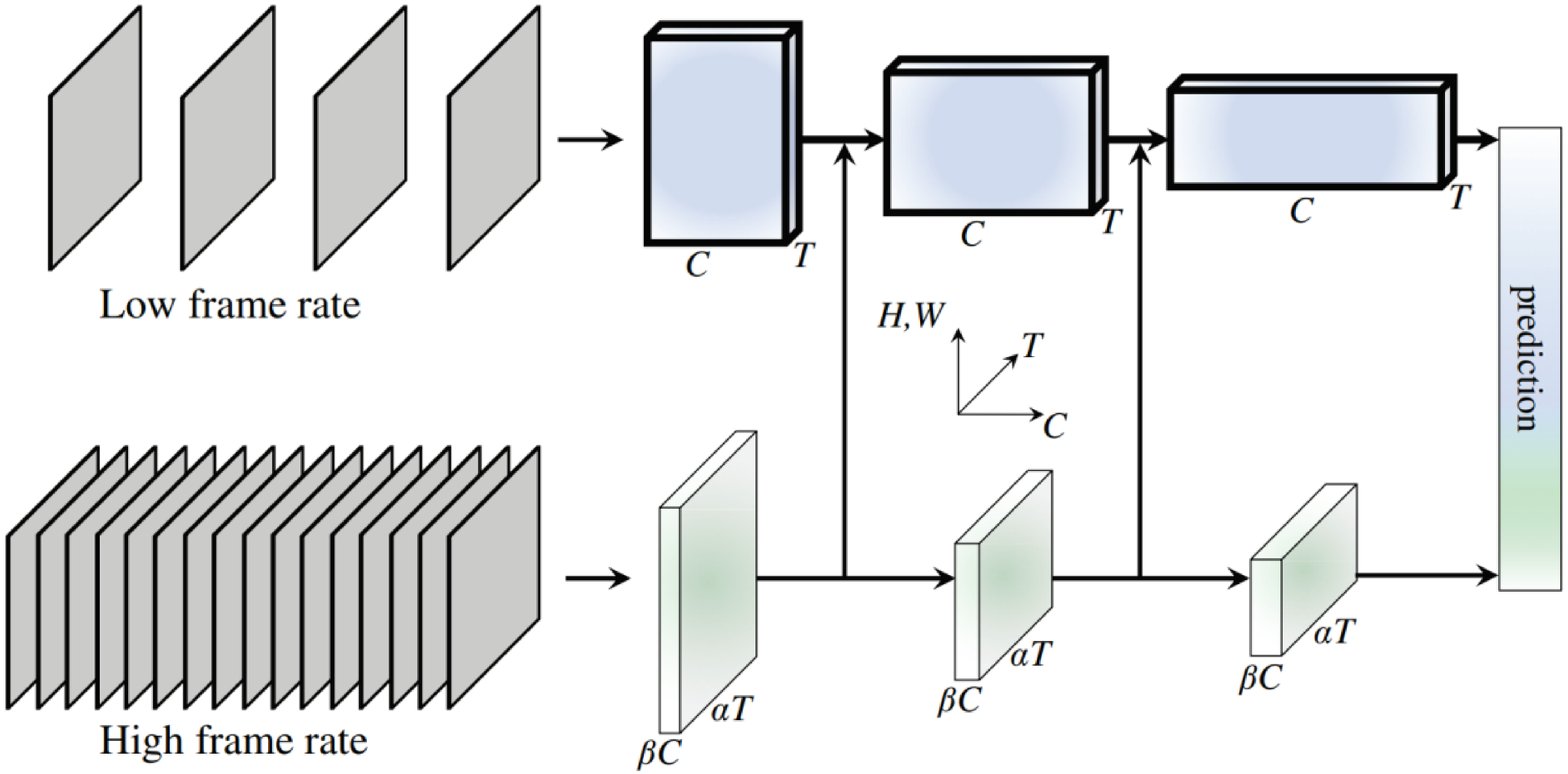



Figure 13: A SlowFast network has a low frame rate, low temporal resolution *Slow* pathway and a high frame rate, $\alpha\times$ higher temporal resolution *Fast* pathway. The Fast pathway is lightweight by using a fraction ($\beta$, e.g., $1/8$) of channels. Lateral connections fuse them.

# 9.6 SlowFast Networks

- the idea of SlowFast is generic, and it can be instantiated with different backbones and implementation specifics
- here, we describe the instantiations of the network architectures
- an example SlowFast model is specified in Figure 14
- we denote spatio-temporal size by $T \times S^2$, where $T$ is the temporal length and $S$ is the height and width of a square spatial crop
- the details are described next

# 9.6 SlowFast Networks

- **Slow pathway**: the Slow pathway in Figure 14 is a temporally strided 3D ResNet
- it has $T = 4$ frames as the network input, sparsely sampled from a 64-frame raw clip with a temporal stride $\tau = 16$
- temporal downsampling is not performed in this instantiation, as doing so would be detrimental when the input stride is large
- unlike typical C3D / I3D models, non-degenerate temporal convolutions (temporal kernel size $> 1$, underlined in Figure 14) are used only in $\text{res}_4$ and $\text{res}_5$; all filters from $\text{conv}_1$ to $\text{res}_3$ are essentially 2D convolution kernels in this pathway
- this is motivated by the experimental observation that using temporal convolutions in earlier layers degrades accuracy
- this is because, when objects move fast and the temporal stride is large, there is little correlation within a temporal receptive field unless the spatial receptive field is large enough (i.e., in later layers)

# 9.6 SlowFast Networks

- **Fast pathway**: Figure 14 shows an example of the Fast pathway with $\alpha = 8$ and $\beta = 1/8$
- it has a much higher temporal resolution (green) and lower channel capacity (orange)
- the Fast pathway has non-degenerate temporal convolutions in every block
- this is motivated by the observation that this pathway holds fine temporal resolution for the temporal convolutions to capture detailed motion
- further, the Fast pathway has no temporal downsampling layers by design

# 9.6 SlowFast Networks

- **Lateral connections**: the lateral connections fuse from the Fast to the Slow pathway
- it requires to match the sizes of features before fusing
- denoting the feature shape of the Slow pathway as $\{T, S^2, C\}$, the feature shape of the Fast pathway is $\{\alpha T, S^2, \beta C\}$
- the following transformations in the lateral connections are experimented with:
  1. (i) Time-to-channel: $\{\alpha T, S^2, \beta C\}$ is reshaped and transposed into $\{T, S^2, \alpha\beta C\}$, meaning that all $\alpha$ frames are packed into the channels of one frame.
  2. (ii) Time-strided sampling: one out of every $\alpha$ frames is sampled, so $\{\alpha T, S^2, \beta C\}$ becomes $\{T, S^2, \beta C\}$.
  3. (iii) Time-strided convolution: a 3D convolution of a $5 \times 1^2$ kernel with $2\beta C$ output channels and stride $= \alpha$ is performed.
- the output of the lateral connections is fused into the Slow pathway by summation or concatenation

# 9.6 SlowFast Networks

| stage | *Slow* pathway | *Fast* pathway | output sizes $T \times S^2$ |
|---|---|---|---|
| raw clip | - | - | $64 \times 224^2$ |
| data layer | stride 16, $1^2$ | stride **2**, $1^2$ | *Slow* : $4 \times 224^2$<br>*Fast* : $32 \times 224^2$ |
| $\text{conv}_1$ | $1 \times 7^2$, 64<br>stride 1, $2^2$ | $5 \times 7^2$, 8<br>stride 1, $2^2$ | *Slow* : $4 \times 112^2$<br>*Fast* : $32 \times 112^2$ |
| $\text{pool}_1$ | $1 \times 3^2$ max<br>stride 1, $2^2$ | $1 \times 3^2$ max<br>stride 1, $2^2$ | *Slow* : $4 \times 56^2$<br>*Fast* : $32 \times 56^2$ |
| $\text{res}_2$ | $\begin{bmatrix} 1 \times 1^2, 64 \\ 1 \times 3^2, 64 \\ 1 \times 1^2, 256 \end{bmatrix} \times 3$ | $\begin{bmatrix} 3 \times 1^2, 8 \\ 1 \times 3^2, 8 \\ 1 \times 1^2, 32 \end{bmatrix} \times 3$ | *Slow* : $4 \times 56^2$<br>*Fast* : $32 \times 56^2$ |
| $\text{res}_3$ | $\begin{bmatrix} 1 \times 1^2, 128 \\ 1 \times 3^2, 128 \\ 1 \times 1^2, 512 \end{bmatrix} \times 4$ | $\begin{bmatrix} 3 \times 1^2, 16 \\ 1 \times 3^2, 16 \\ 1 \times 1^2, 64 \end{bmatrix} \times 4$ | *Slow* : $4 \times 28^2$<br>*Fast* : $32 \times 28^2$ |
| $\text{res}_4$ | $\begin{bmatrix} 3 \times 1^2, 256 \\ 1 \times 3^2, 256 \\ 1 \times 1^2, 1024 \end{bmatrix} \times 6$ | $\begin{bmatrix} 3 \times 1^2, 32 \\ 1 \times 3^2, 32 \\ 1 \times 1^2, 128 \end{bmatrix} \times 6$ | *Slow* : $4 \times 14^2$<br>*Fast* : $32 \times 14^2$ |
| $\text{res}_5$ | $\begin{bmatrix} 3 \times 1^2, 512 \\ 1 \times 3^2, 512 \\ 1 \times 1^2, 2048 \end{bmatrix} \times 3$ | $\begin{bmatrix} 3 \times 1^2, 64 \\ 1 \times 3^2, 64 \\ 1 \times 1^2, 256 \end{bmatrix} \times 3$ | *Slow* : $4 \times 7^2$<br>*Fast* : $32 \times 7^2$ |
| global average pool, concate, fc | | | # classes |

Figure 14: An example instantiation of the SlowFast network. The dimensions of kernels are denoted by $\{T \times S^2, C\}$ for temporal, spatial, and channel sizes. Strides are denoted as {temporal stride, spatial stride$^2$}. Here the speed ratio is $\alpha = 8$ and the channel ratio is $\beta = 1/8$. $\tau$ is 16. The green colors mark *higher* temporal resolution, and orange colors mark *fewer* channels, for the Fast pathway. Non-degenerate temporal filters are underlined. Residual blocks are shown by brackets. The backbone is ResNet-50.

Vă mulţumesc!